

CREACIÓN DEL PROYECTO FRONTEND



Guía Paso a Paso: Listado de Personas con Angular y Servicios

Introducción

En esta lección aprenderemos a conectar nuestra aplicación Angular con los servicios web para mostrar un listado de personas. Configuraremos el servicio en el archivo principal, crearemos un componente que gestione las operaciones con las personas, y nos aseguraremos de que los datos se visualicen correctamente en pantalla. Todo esto se logrará siguiendo las mejores prácticas y usando las herramientas modernas que Angular nos proporciona.

Paso 1: Registrar los servicios en el archivo principal

Descripción:

<https://www.globalmentoring.com.mx>

Vamos a registrar el servicio de `HttpClient` para que esté disponibles en toda la aplicación.

Archivo y Ruta:

`src/app/app.config.ts`

Código:

```
import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';

import { routes } from './app.routes';
import { provideHttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient() // HttpClient
  ]
};
```

Explicación:

Aquí importamos y registramos los servicios necesarios para que puedan ser inyectados en cualquier componente. También se incluye `provideHttpClient` para permitir el uso de `HttpClient`.

Paso 2: Configurar el archivo de rutas para mostrar el componente

Descripción:

Agregamos la ruta raíz ("") para que apunte al componente de personas.

Archivo y Ruta:

`src/app/app.routes.ts`

Código:

```
import { Routes } from '@angular/router';
import { PersonasComponent } from '../personas/personas.component';

export const routes: Routes = [
  { path: '', component: PersonasComponent }
];
```

✚ Explicación:

- Al ingresar a la raíz de la aplicación, se cargará automáticamente el componente `PersonasComponent`.

✚ Paso 3: Asegurarse de tener el router activo

📄 Descripción:

En la clase principal de la aplicación (`app.component.ts`), debe existir el import del módulo `RouterModule` para poder utilizar `<router-outlet>` en la plantilla respectiva.

📁 Archivo y Ruta:

`src/app/app.component.ts`

💻 Código:

```
import { Component } from '@angular/core';
import { RouterModule } from '@angular/router';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  imports: [RouterModule] // Aquí va RouterModule
})
export class AppComponent {
  title = 'personas-frontend-angular';
}
```

✚ Paso 4: Asegurarse de tener el router activo

Descripción:

En la plantilla principal de la aplicación (`app.component.html`), debe existir `<router-outlet>` para mostrar los componentes según la ruta.

Archivo y Ruta:

`src/app/app.component.html`

Código:

```
<div class="box" id="heading">
  <h1>Listado de Personas</h1>
</div>
<router-outlet></router-outlet>
```

Paso 5: Crear el componente `personas`

Descripción:

Generamos el componente que se encargará de mostrar el listado de personas.

Comando utilizado:

```
ng g component personas --skip-tests --inline-style=true
```

Ruta esperada:

`src/app/personas/personas.component.ts`

Código generado automáticamente:

Este archivo ya fue creado con Angular CLI y no se le agregaron estilos ni archivo de pruebas (`--skip-tests, --inline-style=true`).

Paso 6: Configurar el componente `personas` para mostrar datos

Descripción:

Este componente se encargará de mostrar la lista de personas y administrar la lógica para obtener los datos desde el backend.

Archivo y Ruta:

src/app/personas/personas.component.ts

Código:

```
import { Component } from '@angular/core';
import { Persona } from '../persona.model';
import { PersonaService } from '../persona-service';
import { ActivatedRoute, Router } from '@angular/router';

@Component({
  selector: 'app-personas',
  imports: [],
  templateUrl: './personas.component.html'
})
export class PersonasComponent {
  personas: Persona[] = [];

  constructor(private personaService: PersonaService,
    private router: Router,
    private route: ActivatedRoute
  ) { }

  ngOnInit(): void{
    this.personaService.obtenerPersonas()
      .subscribe(
        (personasObtenidas: Persona[]) => {
          //cargamos los datos de persona obtenidos en el arreglo local
          this.personas = personasObtenidas;
          this.personaService.setPersonas(this.personas);
          console.log('personas obtenidas del subscriber:' + this.personas);
        }
      );
  }
}
```

Explicación:

- Se utiliza `ngOnInit()` para obtener los datos al cargar el componente.
- Se inicializa un arreglo de tipo `Persona` que será llenado desde el servicio.

Paso 7: Mostrar las personas en la plantilla HTML

Descripción:

Agregamos la lógica de presentación para mostrar cada persona del arreglo.

Archivo y Ruta:

src/app/personas/personas.component.html

Código:

```
<div class="box">
  @for (persona of personas; track persona.idPersona) {
    <div class="item">
      <ul>
        <li>
          {{ persona.idPersona }} {{ persona.nombre }}
        </li>
      </ul>
    </div>
  }
</div>
```

Explicación:

- Usamos la nueva sintaxis `@for` en lugar de `*ngFor`, disponible desde Angular 17+.
- Mostramos cada persona en un `div.item` con su `idPersona` y `nombre`.

Paso 8: Modificar el Servicio de Personas

Descripción:

Agregamos el método `obtenerPersonas` de `persona-service.ts`

Archivo y Ruta:

src/app/persona-service.html

 Código:

```
obtenerPersonas() {  
    return this.dataService.cargarPersonas();  
}
```

 Explicación:

- ◆ Este método actúa como un **punto entre el componente y el servicio de acceso a datos (DataService)**.
- ◆ No realiza ningún procesamiento por sí mismo, simplemente delega la tarea de obtener el listado de personas al método `cargarPersonas()` del `DataService`.
- ◆ Devuelve un `Observable<Persona[]>`, por lo que debe ser consumido mediante `.subscribe()` en el componente que lo utiliza (como en `PersonasComponent`).

 ¿Por qué se hace así? **Mantiene una separación clara de responsabilidades:**

- `PersonaService` administra la lógica de negocio relacionada con personas.
- `DataService` se encarga de las llamadas HTTP al backend.
 -  Permite **centralizar cambios futuros** (por ejemplo, si se quiere agregar un filtro o una transformación de datos antes de entregarlos al componente).



Paso 9: Modificar el servicio de datos



Descripción:

Modificamos el método `cargarPersonas` para regresar un tipo de datos específico, que es el arreglo de personas.



Archivo y Ruta:

`src/app/data-service.component.html`



Código:

```
cargarPersonas(): Observable<Persona[]> {  
    return this.httpClient.get<Persona[]>(this.urlBase);  
}
```

✓ Cambios realizados:

- ◆ Se especificó el **tipo de retorno del método** como `Observable<Persona[]>`, lo cual antes no estaba declarado explícitamente.
- ◆ También se indicó a `HttpClient` que la respuesta será de tipo `Persona[]`, mediante la sintaxis:

✗ ¿Por qué es importante este cambio?

- ✓ **Mejora el tipado y la seguridad** del código: ahora TypeScript puede verificar que los datos recibidos realmente son un arreglo de objetos `Persona`.
- ✓ Permite **autocompletado y validación** en el resto de la aplicación (por ejemplo, en `subscribe` ya no da error al recibir `Persona[]`).
- ✓ Es una **buena práctica en Angular** dejar explícito el tipo de dato esperado, especialmente cuando se trabaja con peticiones HTTP y observables.

Sección Final: Código completo por archivo trabajado

Código archivo `src/app/persona-service.ts`

```
import { Injectable } from '@angular/core';
import { Persona } from '../persona.model';
import { DataService } from '../data-service';

@Injectable({
  providedIn: 'root'
})
export class PersonaService {

  personas: Persona[] = [];

  constructor(private dataService: DataService) {}

  setPersonas(personas: Persona[]) {
    this.personas = personas;
  }

  obtenerPersonas() {
    return this.dataService.cargarPersonas();
  }
}
```



```

agregarPersona(persona: Persona) {
  console.log('persona a agregar:' + persona.nombre);
  this.dataService.agregarPersona(persona)
    .subscribe({
      next: (persona: Persona) => {
        console.log('se agrega al arreglo la persona recién insertada suscribir:' + persona.idPersona);
        this.personas.push(persona);
      },
      error: (error) => {
        console.error('Error al agregar persona:', error);
      }
    });
}

encontrarPersona(id: number){
  const persona = this.personas.find(p => p.idPersona == id);
  console.log('persona encontrada:' + persona?.idPersona + ' ' + persona?.nombre);
  return persona;
}

modificarPersona(id: number, persona: Persona) {
  console.log('persona a modificar:' + persona.idPersona);
  this.dataService.modificarPersona(id, persona);
}

eliminarPersona(id: number) {
  console.log('eliminar persona con id:' + id);
  const index = this.personas.findIndex(p => p.idPersona == id);
  this.personas.splice(index, 1);
  this.dataService.eliminarPersona(id);
}
}

```



Código archivo `src/app/data-service.ts`

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Persona } from '../persona.model';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})

```

```
    })
    export class DataService {

        constructor(private httpClient: HttpClient) {}

        urlBase = 'http://localhost:8080/personas-backend-java/webservice/personas';

        cargarPersonas(): Observable<Persona[]> {
            return this.httpClient.get<Persona[]>(this.urlBase);
        }

        agregarPersona(persona: Persona): Observable<Persona> {
            return this.httpClient.post<Persona>(this.urlBase, persona);
        }

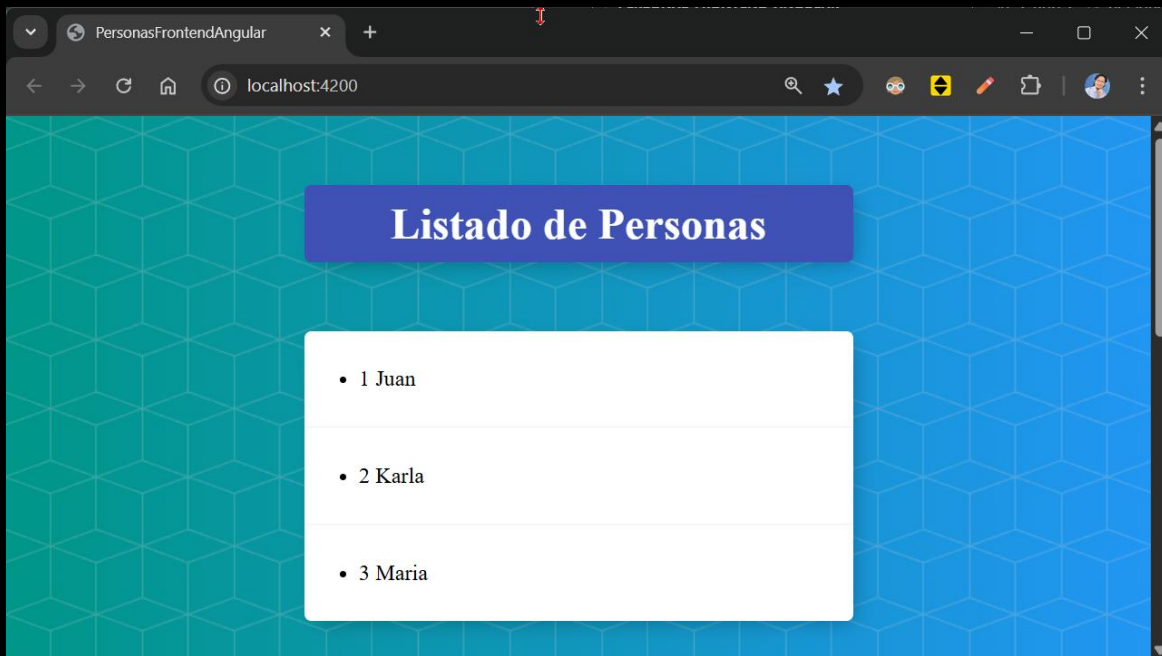
        modificarPersona(idPersona: number, persona: Persona) {
            const url = `${this.urlBase}/${idPersona}`;
            this.httpClient.put(url, persona)
                .subscribe({
                    next: (response) => {
                        console.log('resultado modificar persona:', response);
                    },
                    error: (error) => {
                        console.log('Error en modificar persona:', error);
                    }
                });
        }

        eliminarPersona(idPersona: number) {
            const url = `${this.urlBase}/${idPersona}`;
            this.httpClient.delete(url)
                .subscribe({
                    next: (response) => {
                        console.log('resultado eliminar persona:', response);
                    },
                    error: (error) => {
                        console.log('Error en eliminar persona:', error);
                    }
                });
        }
    }
}
```

✓ Ejecución de la aplicación

Volvemos a ejecutar la aplicación:

- ◆ Detenemos la aplicación, para ello vamos a la consola donde se está ejecutando la aplicación y presionamos `ctl + c`
- ◆ Volvemos a ejecutar el comando: `ng serve -o` para levantar nuevamente la aplicación
- ◆ Entramos a la ruta: <http://localhost:4200/>



✓ Conclusión

En esta lección aprendimos a:

- ◆ Registrar servicios globalmente en Angular con `provideHttpClient`
- ◆ Crear y configurar un componente para mostrar una lista de personas
- ◆ Integrar correctamente el `router-outlet` y la navegación por rutas
- ◆ Utilizar la nueva directiva `@for` para iterar sobre listas

Gracias a esta estructura modular y moderna, nuestra aplicación ya puede conectarse al backend y mostrar los datos en pantalla 🎉.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)